

# Chain-Specific Execution Delay Audit



**September 9, 2026**

*This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.*

Copyright © 2026 Zeppelin Group Ltd.

# Table of Contents

|                                                                                         |                   |
|-----------------------------------------------------------------------------------------|-------------------|
| Table of Contents                                                                       | 2                 |
| Summary                                                                                 | 3                 |
| Scope                                                                                   | 4                 |
| System Overview                                                                         | 5                 |
| Security Model and Trust Assumptions                                                    | 5                 |
| <a href="#">Low Severity</a>                                                            | <a href="#">7</a> |
| <a href="#">L-01 Node May Not Observe Per-Chain Execution Delay</a>                     | <a href="#">7</a> |
| Notes & Additional Information                                                          | 7                 |
| <a href="#">N-01 Undocumented Interaction Between Execution Delay and Priority Mode</a> | <a href="#">7</a> |
| Conclusion                                                                              | 9                 |
| Appendix                                                                                | 10                |
| Issue Classification                                                                    | 10                |

# Summary

|           |                                  |                                |                |
|-----------|----------------------------------|--------------------------------|----------------|
| Type      | Layer 2 & Rollups                | Total Issues                   | 2 (1 resolved) |
| Timeline  | From 2026-09-01<br>To 2026-09-01 | Critical Severity Issues       | 0 (0 resolved) |
| Languages | Solidity                         | High Severity Issues           | 0 (0 resolved) |
|           |                                  | Medium Severity Issues         | 0 (0 resolved) |
|           |                                  | Low Severity Issues            | 1 (1 resolved) |
|           |                                  | Notes & Additional Information | 1 (0 resolved) |
|           |                                  | Client Reported Issues         | 0 (0 resolved) |

# Scope

OpenZeppelin performed an audit of the [pull request #2439](#) at the [a73bb8a](#) commit in the [matter-labs/era-contracts](#) repository.

In scope were the following files:

```
l1-contracts/  
├── contracts/  
│   ├── common/  
│   │   ├── Config.sol  
│   │   └── L1ContractErrors.sol  
│   └── state-transition/  
│       ├── L1StateTransitionErrors.sol  
│       └── validators/  
│           ├── EraMultisigValidator.sol  
│           ├── ValidatorTimelock.sol  
│           └── interfaces/  
│               └── IValidatorTimelock.sol
```

Subsequently we also reviewed [pull request #2484](#) at commit [621447a](#) in the same repository.

# System Overview

PR #2439 extends `ValidatorTimelock` with chain-specific execution delays. The existing ecosystem-wide `executionDelay` remains controlled by the timelock owner and acts as a minimum delay for every chain. A chain admin can independently increase its chain's delay through `increaseChainExecutionDelay()`, while decreasing or resetting a chain-specific delay is also restricted to the ecosystem owner through `setChainExecutionDelay()`. The effective delay is calculated as the maximum of the ecosystem-wide and chain-specific values.

`ValidatorTimelock.executeBatchesSharedBridge()` is updated to use this effective per-chain delay when determining whether committed batches can be executed, making the new configuration directly enforceable in the batch execution path.

Both ecosystem-wide and chain-specific delays are capped at 7 days. This cap is intended to prevent accidental excessive configuration and is explicitly not a trust guarantee, as governance can upgrade the timelock and other administrative actions may still affect chain liveness.

For `EraMultisigValidator`, which does not enforce the timelock itself, the execution-delay getters now read directly from the downstream `ValidatorTimelock`. The corresponding setters are disabled to prevent successful transactions that would modify unused local state without changing the delay actually being enforced.

PR #2484 raises `Config.MAX_VALIDATOR_TIMELOCK_EXECUTION_DELAY` from 7 days to 30 days, so a chain admin can configure an execution delay of up to a month instead of a week.

## Security Model and Trust Assumptions

The chain admin is a privileged role with access to set sensitive configuration:

- The chain admin is assumed to act honestly and in the best interests of the protocol.

- The chain admin is trusted to account for the configured `PRIORITY_EXPIRATION` value when setting the chain's specific `executionDelay`. In particular, `executionDelay` is expected not to exceed `PRIORITY_EXPIRATION` under normal circumstances, as doing so would allow users to trivially activate priority mode, effectively transferring validation responsibility to the PermissionlessValidator.

# Low Severity

## L-01 Node May Not Observe Per-Chain Execution Delay

In `ValidatorTimelock`, the `executionDelay` variable continues to expose the ecosystem-wide floor, while the `getExecutionDelay` function is introduced as the chain-specific enforced value in `executeBatchesSharedBridge`.

The `eth_sender` transaction aggregator [appears to read the ecosystem-wide getter](#), and the node documentation [describes](#) that call as the source of the delay it uses.

If that is the value used to schedule execute transactions, then a chain that raises its delay through `ValidatorTimelock.increaseChainExecutionDelay` would have its node schedule execution too early, and the resulting transactions would revert with `TimeNotReached` until the enforced delay elapses.

Consider confirming whether the node should read `getExecutionDelay(_chainAddress)` instead, and whether the corresponding server-side change is already tracked before per-chain delays are enabled on any chain.

**Update:** Resolved in the [zksync-era-private](#) repository in [pull request #101](#).

## Notes & Additional Information

### N-01 Undocumented Interaction Between Execution Delay and Priority Mode

`ValidatorTimelock.increaseChainExecutionDelay` allows a chain admin to raise its chain's execution delay up to the `MAX_EXECUTION_DELAY` of 7 days.

Separately, [anyone can activate Priority Mode](#) once the oldest unprocessed priority transaction is older than the `PRIORITY_EXPIRATION` of 4 days.

These two settings interact. A priority transaction only counts as processed once the batch containing it is executed, and execution is what the delay holds back:

- The [delay gates execution](#).
- The unprocessed index advances only in `PriorityTree.processBatch`, whose single call site is inside `Executor._executeOneBatch`.
- Activation [compares](#) that index's request timestamp against `PRIORITY_EXPIRATION`.

If a chain sets its delay above 4 days, priority transactions remain unprocessed past the 4-day mark, no matter how promptly the operator commits. Consequently, anyone can activate Priority Mode against a fully caught-up, honest operator. Activation then reverts all committed-but-unexecuted batches and restricts settlement to the `PermissionlessValidator` until the `ChainTypeManager` owner deactivates it.

This issue affects chains that have called `permanentlyAllowPriorityMode` and are [permanent rollups](#).

Consider adding a warning to the [docstring of `MAX\_VALIDATOR\_TIMELOCK\_EXECUTION\_DELAY`](#) in `Config.sol`, stating that for chains with Priority Mode enabled, an enforced execution delay above `PRIORITY_EXPIRATION` makes Priority Mode permanently activatable by anyone, and that such chains should keep their delay below `PRIORITY_EXPIRATION`.

**Update:** Acknowledged, will resolve.



# Conclusion

PR #2439 introduces chain-specific execution delays to `ValidatorTimelock`, allowing individual chains to enforce stricter execution delays while preserving the ecosystem-wide delay as a minimum.

No serious issues were identified during the review.

We are grateful to the Matter Labs team for their collaboration and responsiveness throughout the engagement.

# Appendix

## Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

### Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

### High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

### Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

### **Low Severity**

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

### **Notes & Additional Information Severity**

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.